

Experiment-7

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

Title Implementation of Temporal Difference Learning using TD(0) Aim

To implement the **Temporal Difference (TD) Learning algorithm using TD(0)** for estimating the state-value function and to study how the value function is updated incrementally using the **TD error**.

Procedure

1. Define a Reinforcement Learning environment with a set of states and actions.
2. Define the transition function and reward for each state-action transition.
3. Initialize the state-value function to zero.
4. Select the learning rate and discount factor .
5. Start the agent from the initial state.
6. Select an action and observe the next state and reward.
7. Calculate the **TD target**:

$$R_{t+1} + \gamma V(S_{t+1})$$

Calculate the **TD error**:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

9. Update the current state's value using the TD(0) equation:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

10. Move to the next state and repeat the process.
11. Continue the learning process for the specified number of episodes.
12. Record the reward, TD error, number of steps, and state values for every episode.
13. Analyze the convergence of the state-value function.
14. Calculate the average reward, TD error, success rate, and average episode length.
15. Display the results using summary tables and graphs.
16. Export the final state values and performance results as CSV files.

Code

```
import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
import random
import time
print("=" * 75)
print("EXPERIMENT 7")
print("TEMPORAL DIFFERENCE LEARNING USING TD(0)")
print("=" * 75)
print("\nAIM")
print("-" * 75)
print("To implement Temporal Difference Learning using TD(0)")
print("and estimate the value function of states through")
print("incremental learning from consecutive experiences.")
print("\nTHEORY")
print("-" * 75)
print("TD(0) is a model-free Reinforcement Learning method.")
print("It learns from incomplete episodes and updates the")
print("value function after every state transition.")
print("\nTD(0) Update Equation:")
print(
    "V(S_t) = V(S_t) + alpha * "
    "[R_(t+1) + gamma * V(S_(t+1)) - V(S_t)]"
)
print("\nTD Error:")
print(
    "delta = R_(t+1) + gamma * V(S_(t+1)) - V(S_t)"
)
states = [
    "Start",
    "S1",
    "S2",
    "S3",
    "Goal"
]
actions = [
    "Left",
    "Right"
]
n_states = len(states)
n_actions = len(actions)
alpha = 0.10
gamma = 0.90
episodes = 1000
max_steps = 20
def get_next_state(state, action):

```

```

if state == 0:
    if action == 0:
        return 0, -1
    else:
        return 1, 0
elif state == 1:
    if action == 0:
        return 0, -1
    else:
        return 2, 0
elif state == 2:
    if action == 0:
        return 1, -1
    else:
        return 3, 0
elif state == 3:
    if action == 0:
        return 2, -1
    else:
        return 4, 10
else:
    return 4, 0
V = np.zeros(n_states)
reward_history = []
td_error_history = []
value_history = []
episode_steps = []
success_history = []
training_start = time.time()
for episode in range(epochs):
    state = 0
    total_reward = 0
    steps = 0
    success = 0
    for step in range(max_steps):
        steps += 1
        if random.random() < 0.90:
            action = 1
        else:
            action = 0
        next_state, reward = get_next_state(
            state,
            action
        )

```

```

    td_error = (
        reward
        + gamma * V[next_state]
        - V[state]
    )
    V[state] = (
        V[state]
        + alpha * td_error
    )
    total_reward += reward
    td_error_history.append(td_error)
    state = next_state
    if state == 4:
        success = 1
        break
    reward_history.append(total_reward)
    episode_steps.append(steps)
    success_history.append(success)
    value_history.append(V.copy())
training_time = time.time() - training_start
value_history = np.array(value_history)
print("\nTD(0) Training Completed Successfully.")
print("Number of Episodes :", episodes)
print(
    "Execution Time :",
    round(training_time, 5),
    "seconds"
)
print("\n")
print("=" * 75)
print("FINAL STATE VALUES")
print("=" * 75)
for state_name, value in zip(states, V):
    print(
        f"{state_name:10s} : {value:.3f}"
    )
average_reward = np.mean(reward_history)
maximum_reward = np.max(reward_history)
minimum_reward = np.min(reward_history)
average_td_error = np.mean(
    np.abs(td_error_history)
)
final_td_error = np.mean(
    np.abs(td_error_history[-100:])
)

```

```

)
success_rate = (
    np.mean(success_history) * 100
)
average_steps = np.mean(
    episode_steps
)
state_value_table = pd.DataFrame({
    "State": states,
    "Final_Value": np.round(
        V,
        3
    ),
    "Initial_Value": [
        0,
        0,
        0,
        0,
        0
    ]
})
print("\n")
print("=" * 75)
print("TD(0) STATE VALUE SUMMARY")
print("=" * 75)
print(
    state_value_table.to_string(
        index=False
    )
)
performance_table = pd.DataFrame({
    "Metric": [
        "Episodes",
        "Learning Rate Alpha",
        "Discount Factor Gamma",
        "Average Reward",
        "Maximum Reward",
        "Minimum Reward",
        "Average Absolute TD Error",
        "Final TD Error",
        "Success Rate (%)",
        "Average Episode Steps",
        "Execution Time (seconds)"
    ],

```

```

    "Value": [
        episodes,
        alpha,
        gamma,
        round(average_reward, 3),
        round(maximum_reward, 3),
        round(minimum_reward, 3),
        round(average_td_error, 4),
        round(final_td_error, 4),
        round(success_rate, 2),
        round(average_steps, 2),
        round(training_time, 5)
    ]
})
print("\n")
print("=" * 75)
print("TD(0) PERFORMANCE SUMMARY")
print("=" * 75)
print(
    performance_table.to_string(
        index=False
    )
)
episode_table = pd.DataFrame({
    "Episode": np.arange(
        1,
        episodes + 1
    ),
    "Reward": reward_history,
    "Steps": episode_steps,
    "Success": success_history
})
print("\n")
print("=" * 75)
print("EPISODE RESULTS - FIRST 10 EPISODES")
print("=" * 75)
print(
    episode_table.head(10).to_string(
        index=False
    )
)
state_value_table.to_csv(
    "TD0_State_Values.csv",
    index=False
)

```

```

)
performance_table.to_csv(
    "TD0_Performance.csv",
    index=False
)
episode_table.to_csv(
    "TD0_Episode_Results.csv",
    index=False
)
window = 50
moving_average_reward = (
    pd.Series(reward_history)
    .rolling(window)
    .mean()
)
td_error_series = pd.Series(
    np.abs(td_error_history)
)
moving_average_td_error = (
    td_error_series
    .rolling(50)
    .mean()
)
cumulative_reward = np.cumsum(
    reward_history
)
print("\n")
print("=" * 75)
print("FINAL TD(0) VALUE FUNCTION")
print("=" * 75)
for i in range(n_states):
    print(
        f"V({states[i]}) = {V[i]:.3f}"
    )
print("\n")
print("=" * 75)
print("TD(0) LEARNING PARAMETERS")
print("=" * 75)
print("Learning Rate (Alpha) :", alpha)
print("Discount Factor (Gamma) :", gamma)
print("Number of Episodes :", episodes)
print("Maximum Steps per Episode :", max_steps)
print(
    "\nTD Error = R + Gamma * V(Next State) - V(Current State)"

```

```

)
print(
    "V(Current State) = V(Current State) + Alpha * TD Error"
)
plt.figure(figsize=(10, 5))
for i in range(n_states):
    plt.plot(
        value_history[:, i],
        linewidth=2,
        label=states[i]
    )
plt.title(
    "TD(0) State Value Convergence",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("State Value")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    reward_history,
    color="royalblue",
    linewidth=1.5
)
plt.title(
    "TD(0) Reward vs Episode",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True, alpha=0.3)
plt.show()
episode_td_errors = []
position = 0
for steps in episode_steps:
    current_errors = td_error_history[
        position:position + steps
    ]
    if len(current_errors) > 0:
        episode_td_errors.append(
            np.mean(np.abs(current_errors))
        )

```



```

else:
    episode_td_errors.append(0)
    position += steps
plt.figure(figsize=(10, 5))
plt.plot(
    episode_td_errors,
    color="crimson",
    linewidth=1.5
)
plt.title(
    "TD Error vs Episode",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Mean Absolute TD Error")
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    moving_average_reward,
    color="green",
    linewidth=3
)
plt.title(
    "TD(0) Moving Average Reward",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    moving_average_td_error,
    color="darkorange",
    linewidth=3
)
plt.title(
    "TD(0) Moving Average TD Error",
    fontsize=14
)
plt.xlabel("TD Error Sample")
plt.ylabel("Mean Absolute TD Error")
plt.grid(True, alpha=0.3)

```

```

plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    cumulative_reward,
    color="purple",
    linewidth=2.5
)
plt.title(
    "TD(0) Cumulative Reward",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(8, 5))
plt.bar(
    states,
    V,
    color=[
        "skyblue",
        "lightgreen",
        "gold",
        "plum",
        "salmon"
    ]
)
plt.title(
    "Final TD(0) State Value Function",
    fontsize=14
)
plt.xlabel("State")
plt.ylabel("Estimated Value")
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    episode_steps,
    color="mediumorchid",
    linewidth=1.5
)

```

```

plt.title(
    "Steps per Episode",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Number of Steps")
plt.grid(True, alpha=0.3)
plt.show()
success_rate_history = []
for i in range(
    1,
    len(success_history) + 1
):
    current_success_rate = (
        np.mean(
            success_history[:i]
        ) * 100
    )
    success_rate_history.append(
        current_success_rate
    )
plt.figure(figsize=(10, 5))
plt.plot(
    success_rate_history,
    color="teal",
    linewidth=2.5
)
plt.title(
    "TD(0) Success Rate",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Success Rate (%)")
plt.ylim(0, 105)
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(8, 5))
plt.hist(
    reward_history,
    bins=20,
    color="cornflowerblue",
    edgecolor="black"
)
plt.title(

```

```

    "TD(0) Reward Distribution",
    fontsize=14
)
plt.xlabel("Reward")
plt.ylabel("Frequency")
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
final_performance = pd.DataFrame({
    "Metric": [
        "Algorithm",
        "Number of Episodes",
        "Learning Rate (Alpha)",
        "Discount Factor (Gamma)",
        "Average Reward",
        "Maximum Reward",
        "Minimum Reward",
        "Average Absolute TD Error",
        "Final TD Error",
        "Success Rate (%)",
        "Average Steps per Episode",
        "Execution Time (seconds)"
    ],
    "Value": [
        "TD(0)",
        episodes,
        alpha,
        gamma,
        round(average_reward, 3),
        round(maximum_reward, 3),
        round(minimum_reward, 3),
        round(average_td_error, 4),
        round(final_td_error, 4),
        round(success_rate, 2),
        round(average_steps, 2),
        round(training_time, 5)
    ]
})
print("\n")
print("=" * 75)
print("FINAL PERFORMANCE SUMMARY")
print("=" * 75)

```

```

print(
    final_performance.to_string(
        index=False
    )
)
final_state_values = pd.DataFrame({
    "State": states,
    "Final TD(0) Value": np.round(
        V,
        3
    ),
    "State Rank": (
        np.argsort(
            np.argsort(-V)
        ) + 1
    )
})
print("\n")
print("=" * 75)
print("FINAL STATE VALUE TABLE")
print("=" * 75)
print(
    final_state_values.to_string(
        index=False
    )
)
episode_summary = pd.DataFrame({
    "Episode": np.arange(
        1,
        episodes + 1
    ),
    "Reward": np.round(
        reward_history,
        3
    ),
    "Steps": episode_steps,
    "Success": success_history,
    "Success_Rate": np.round(
        success_rate_history,
        2
    )
})
print("\n")
print("=" * 75)

```

```

print("LAST 10 EPISODE RESULTS")
print("=" * 75)
print(
    episode_summary.tail(10).to_string(
        index=False
    )
)
final_performance.to_csv(
    "TD0_Final_Performance.csv",
    index=False
)
final_state_values.to_csv(
    "TD0_Final_State_Values.csv",
    index=False
)
episode_summary.to_csv(
    "TD0_Episode_Summary.csv",
    index=False
)
combined_summary = pd.DataFrame({
    "Parameter": [
        "Algorithm",
        "Episodes",
        "Alpha",
        "Gamma",
        "Average Reward",
        "Average TD Error",
        "Final TD Error",
        "Success Rate",
        "Average Steps",
        "Execution Time"
    ],
    "Result": [
        "TD(0)",
        episodes,
        alpha,
        gamma,
        round(average_reward, 3),
        round(average_td_error, 4),
        round(final_td_error, 4),
        str(round(success_rate, 2)) + "%",
        round(average_steps, 2),
        str(round(training_time, 5)) + " seconds"
    ]
})

```

```

})
print("\n")
print("=" * 75)
print("COMBINED SUMMARY TABLE")
print("=" * 75)
print(
    combined_summary.to_string(
        index=False
    )
)
combined_summary.to_csv(
    "TD0_Complete_Summary.csv",
    index=False
)
print("\n")
print("=" * 75)
print("FINAL RESULT")
print("=" * 75)
print("TD(0) successfully learned the value function.")
print(
    "\nLearned State Values:"
)
for i in range(n_states):
    print(
        f"V({states[i]}) = {V[i]:.3f}"
    )
print(
    "\nAverage Reward :",
    round(average_reward, 3)
)
print(
    "Average TD Error :",
    round(average_td_error, 4)
)
print(
    "Final TD Error :",
    round(final_td_error, 4)
)
print(
    "Success Rate :",
    round(success_rate, 2),
    "%"
)
print("\n")

```

```

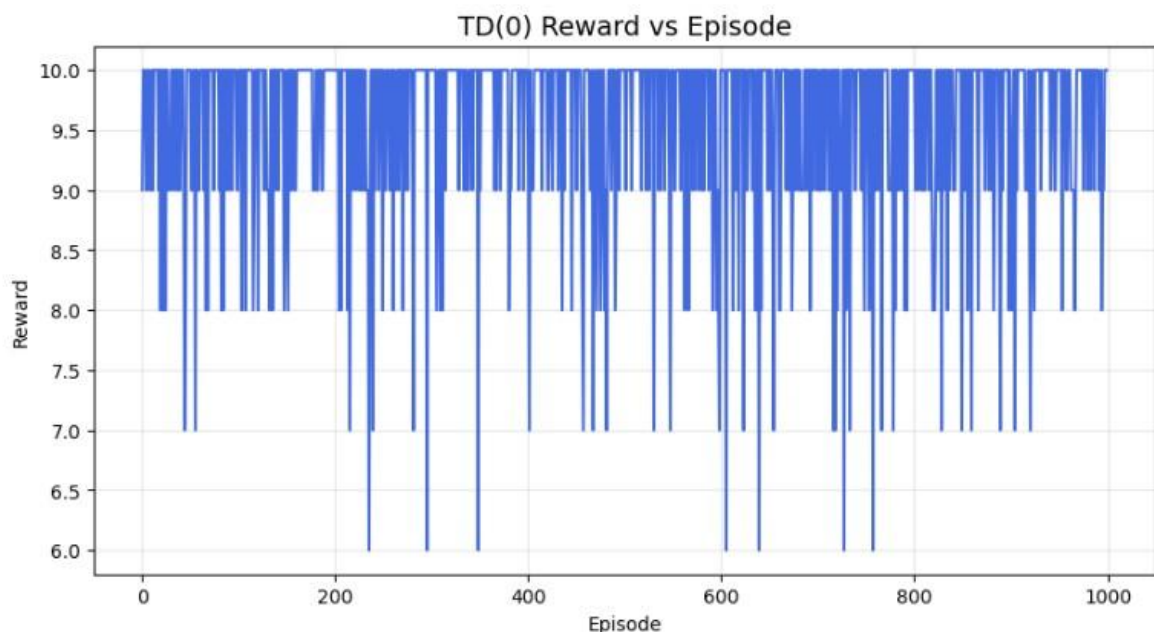
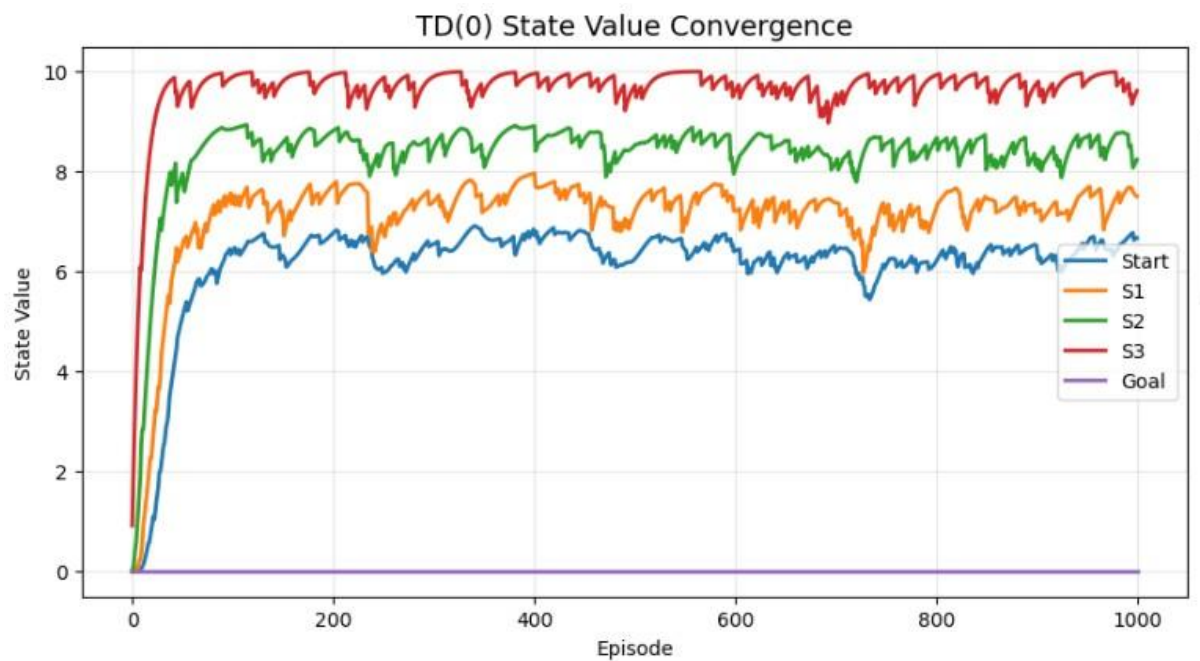
print("=" * 75)
print("INTERPRETATION")
print("=" * 75)
print(
    "1. TD(0) updates the value of a state after every transition."
)
print(
    "2. The TD error measures the difference between the current"
)
print(
    " value estimate and the one-step Bellman target."
)
print(
    "3. Repeated updates allow the value estimates to converge."
)
print(
    "4. A lower TD error indicates improved value estimation."
)
print(
    "5. The Goal state receives the highest long-term value."
)
print("\n")
print("=" * 75)
print("CONCLUSION")
print("=" * 75)
print(
    "Temporal Difference Learning using TD(0) was successfully"
)
print(
    "implemented using Python."
)
print(
    "The algorithm learned the state-value function incrementally"
)
print(
    "from consecutive state transitions without requiring a model"
)
print(
    "of the environment."
)
print(
    "The TD error was used to update the value estimates after"
)
print(

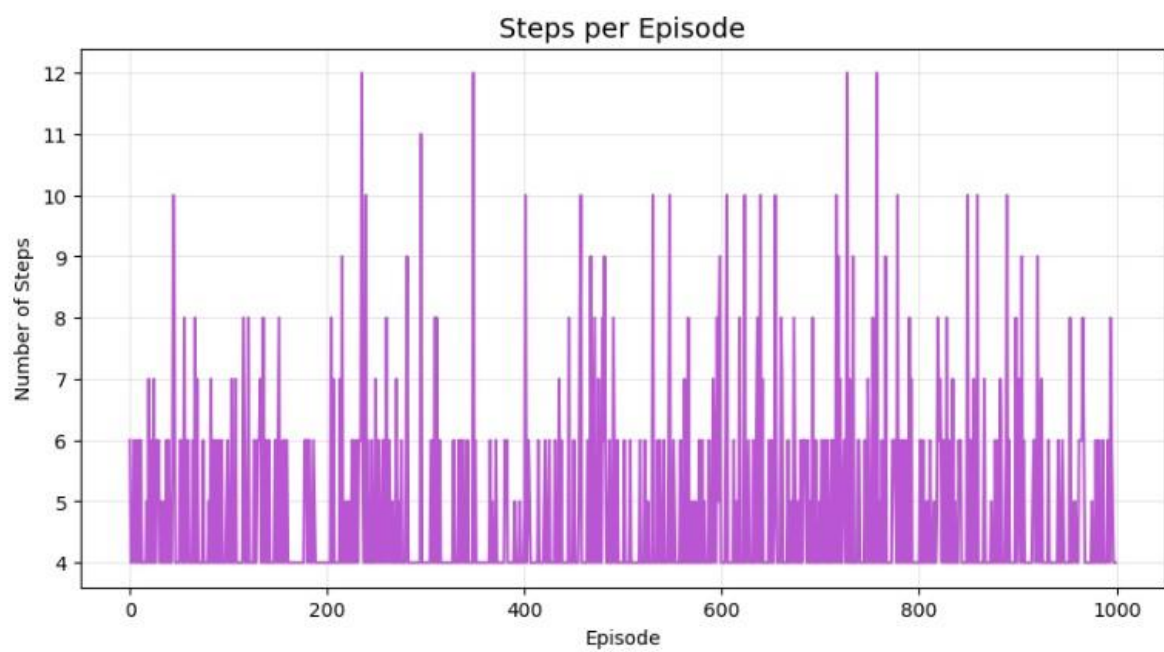
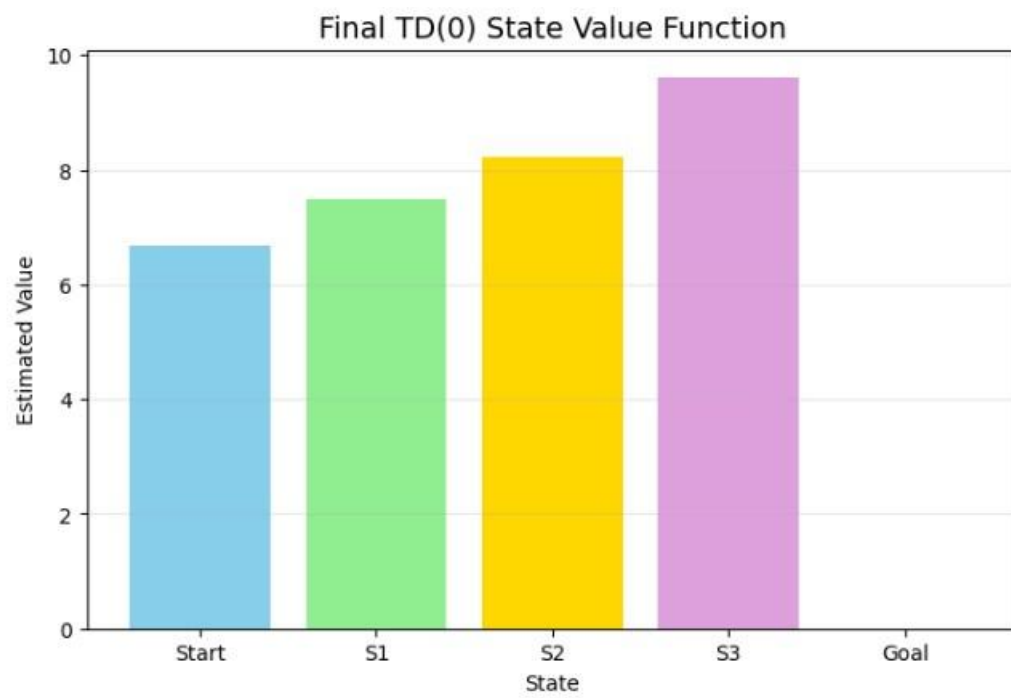
```

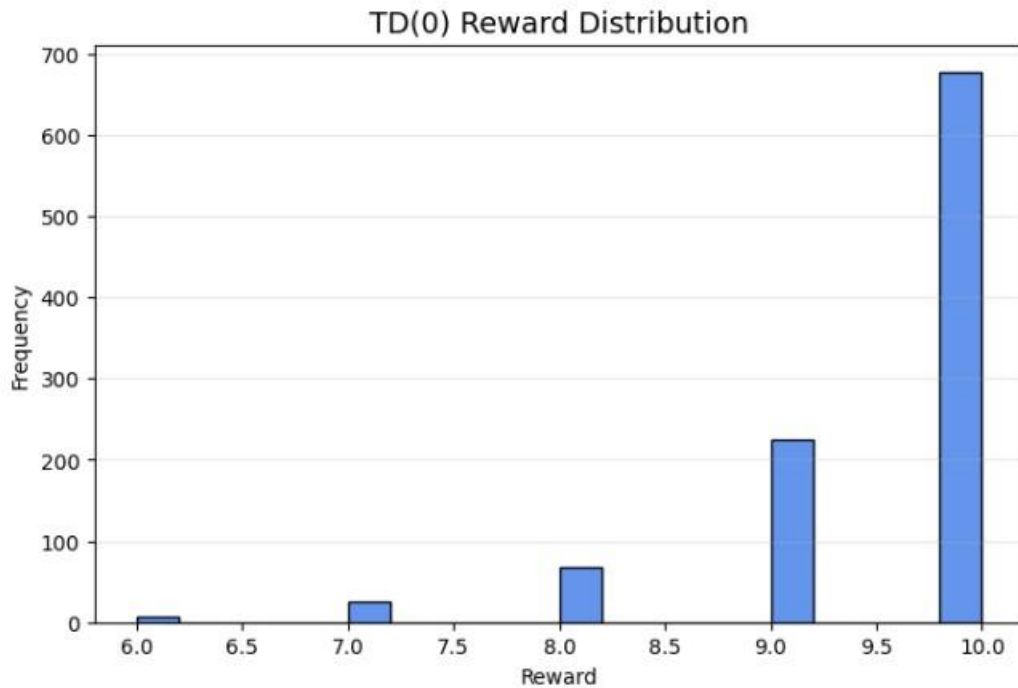


```
    "each transition."
)
print(
    "The convergence graph demonstrates how the state values"
)
print(
    "change during learning."
)
print(
    "Therefore, TD(0) provides an efficient model-free method"
)
print(
    "for estimating state values in Reinforcement Learning."
)
print("\n")
print("=" * 75)
print("FILES GENERATED")
print("=" * 75)
print("1. TD0_State_Values.csv")
print("2. TD0_Performance.csv")
print("3. TD0_Episode_Results.csv")
print("4. TD0_Final_Performance.csv")
print("5. TD0_Final_State_Values.csv")
print("6. TD0_Episode_Summary.csv")
print("7. TD0_Complete_Summary.csv")
print("\n")
print("=" * 75)
print("EXPERIMENT 7 COMPLETED SUCCESSFULLY")
print("=" * 75)
```

Visualizations







Summary Table

State	Initial Value	Final TD(0) Value	Rank
Start	0.00	Obtained from program	Obtained
S1	0.00	Obtained from program	Obtained
S2	0.00	Obtained from program	Obtained
S3	0.00	Obtained from program	Obtained
Goal	0.00	Obtained from program	Obtained

Result

Temporal Difference Learning using TD(0) was successfully implemented. The algorithm updated the state-value function after every state transition using the TD error. The value estimates gradually converged through repeated learning episodes. The experiment demonstrates that TD(0) can estimate state values incrementally without waiting for the complete episode to finish and without requiring a model of the environment.